

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I Shaik Mohammed Waseem Rayan (192373004) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Shaik Mohammed Waseem Rayan

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search

- ii. Analyze the challenges of partial observability and dynamic environments
- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Answers

Scenario 1:

Task 1:

Greedy Best-First Search (GBFS) expands the node whose heuristic value $h(n)$ — here, straight-line distance to the destination — is smallest, and it completely ignores the path cost $g(n)$ already incurred. In this flood scenario, the drone would always move toward whichever waypoint looks geometrically closest to the destination, without weighing how difficult, flooded, or weather-affected that path actually is. Strengths: it is fast and memory-light, which suits the urgency of medicine delivery, since it only needs to evaluate the heuristic rather than accumulate cost. Weaknesses: because it never reconsiders the real cost already spent, it can be lured toward a route that looks short on a straight line but is actually blocked, flooded, or far more expensive in real terrain — leading to detours, dead ends, or even delivery failure. It is neither complete nor optimal, and under uncertainty (partial real-time updates about blockages) it may commit to a route before learning that it is unsafe.

Task 2:

A* evaluates nodes using $f(n) = g(n) + h(n)$, where $g(n)$ is the actual cost accumulated so far (incorporating terrain difficulty and weather penalties) and $h(n)$ is the straight-line estimate to the goal. This balance means A* does not blindly chase the geometrically closest waypoint; it also accounts for how costly the journey so far has been. When real-time updates report a blocked path, the drone can update $g(n)$ for affected edges (or mark them as impassable) and A* will naturally re-prioritise alternative routes whose combined actual-cost-plus-estimate is lower. In dynamic settings this is typically implemented with an incremental/replanning variant (e.g., D* Lite or Anytime Repairing A*) so the drone does not need to search entirely from scratch every time a road becomes inaccessible. As long as the heuristic is admissible (never overestimates true remaining cost), A* guarantees the path found is optimal in terms of true cost, which is far safer than GBFS's purely heuristic-driven choices.

Task 3:

For GBFS, heuristic accuracy is everything, because the heuristic is the sole basis for every decision — an inaccurate heuristic (e.g., straight-line distance across a flooded, impassable region) directly translates into poor, sometimes disastrous, routing choices with no cost-based correction available. For A*, heuristic accuracy affects efficiency and, in one direction, optimality: if the heuristic is admissible and consistent, A* remains optimal regardless of how 'tight' the estimate is, but a weaker (lower) heuristic causes A* to explore more nodes and behave more like Uniform Cost Search, while a heuristic that overestimates true cost breaks the optimality guarantee and can cause A* to return a suboptimal — potentially unsafe — route, similar in spirit to GBFS.

Task 4:

When roads are suddenly blocked, any previously computed route can become invalid instantly. GBFS is particularly vulnerable because it may only discover a route is unusable once the drone physically reaches the blockage, wasting time and increasing risk during an emergency. A*, especially in its incremental replanning forms, can react more gracefully: it updates only the parts of the search graph affected by the new information and re-derives a new optimal path without restarting the entire search, which is critical when delivery windows

are tight. In both cases, however, frequent environmental changes increase computational overhead, since more of the search space must be revisited; the key difference is that A*'s cost-aware bookkeeping makes this re-evaluation far more targeted and reliable than GBFS's heuristic-only approach.

Task 5:

A* (or a dynamic/anytime variant such as D* Lite or Weighted A* with iterative deepening on the time budget) is the recommended approach. It preserves near-optimal, cost-aware routing — essential when 'cost' represents real flight risk and delay in an emergency — while still allowing bounded, time-limited search so the drone can act quickly under urgency. GBFS alone is too risky here because it ignores real incurred cost and blocked-path information, which could send the drone into hazardous or dead-end routes. A weighted/anytime A* offers the best compromise: it behaves close to A* in quality but can be tuned to return a 'good enough' route almost immediately and refine it if time permits, satisfying the scenario's demand for speed, safety, and near-optimal delivery routing simultaneously.

Scenario 1:

Task 1:

Hill Climbing starts from a current signal-timing configuration and repeatedly makes a small local change (e.g., adjusting one intersection's green-light duration), keeping the change only if it reduces the objective function (waiting time, fuel, congestion), and stopping when no neighboring adjustment improves the outcome. This is simple and fast to compute, which is attractive for a real-time system. Its limitations are that it is purely greedy — it never accepts a temporarily worse configuration even if that would eventually lead to a much better one — has no backtracking, is highly sensitive to the starting configuration, and, most critically here, is prone to becoming stuck in local optima where every single-intersection tweak looks unhelpful even though a coordinated change across several intersections would substantially reduce congestion.

Task 2:

Local maxima: a signal-timing configuration where no single-intersection adjustment improves the objective, yet a fundamentally different configuration (for example, a synchronized 'green wave' along a major corridor) would perform far better — Hill Climbing cannot see past this because it only evaluates immediate neighbors. Plateaus: regions of the search space where many nearby configurations yield nearly identical waiting times (e.g., during low-traffic overnight hours, small timing changes barely matter), causing the algorithm to wander without clear improvement signals and potentially stop prematurely. Ridges: cases where the true improving direction requires simultaneously adjusting multiple intersections together (for instance, two adjacent intersections must change their offsets in a coordinated way to create a green wave); because Hill Climbing typically changes one variable at a time, it fails to find this 'diagonal' improving move even though it exists.

Task 3:

Simulated Annealing (SA) introduces controlled randomness: it occasionally accepts a worse configuration with a probability that starts high and gradually decreases according to a cooling schedule. This lets the search escape local maxima and cross plateaus early on, while converging toward a strong solution as the 'temperature' cools — well suited to periodically

re-optimizing signal timings as traffic evolves throughout the day. Genetic Algorithms (GA) instead maintain a population of candidate signal-timing plans; through selection, crossover, and mutation, good partial solutions ('genes' representing timings at particular intersections) from different candidates can be combined, allowing the algorithm to effectively make coordinated, multi-intersection ('ridge-like') improvements that Hill Climbing cannot reach in a single step, and its population-based exploration naturally reduces the risk of any single run getting permanently stuck.

Task 4:

For a city-scale deployment across hundreds of intersections with continuously shifting traffic patterns, a Genetic Algorithm (potentially hybridized with local search, i.e., a memetic algorithm) is recommended. GA's population-based search evaluates many candidate configurations in parallel, scales naturally by distributing intersections/sub-regions across the population's genome, and can be warm-started with the previous best population when re-optimizing for new traffic conditions, giving strong adaptability. Simulated Annealing remains useful as a lightweight, fast local refinement step (e.g., fine-tuning a single intersection or small sub-network) once the GA has located a promising region of the search space, combining GA's global exploration with SA's efficient local exploitation for a scalable, adaptive traffic-signal optimization system.

Scenario 3:

Task 1:

Offline search requires complete knowledge of the state space before acting — the agent computes an entire plan in advance and then executes it blindly. This is infeasible for the rover because the full Martian terrain map is unknown in advance and cannot be transmitted or sensed remotely with sufficient detail. An online search agent instead interleaves computation with action: it senses its immediate surroundings, decides on the next move, acts, observes the outcome, and updates its internal knowledge — repeating this cycle continuously. This is essential here because hazards (craters, loose rocks) are discovered only as the rover physically encounters them, and any offline plan built on incomplete information could lead the rover into danger.

Task 2:

Partial observability means the rover can only sense terrain within its limited sensor range (cameras, LIDAR), so anything beyond that range is unknown and must be assumed or explored before it can be trusted. Dynamic conditions — for example, dust movement or newly exposed hazards — mean that terrain previously judged safe may not remain so, requiring the rover to periodically re-verify rather than rely purely on old data. Compounding this, communication delay between Mars and Earth (several minutes each way) rules out real-time human tele-operation for every decision, so the rover must autonomously balance exploration (mapping the unknown to reduce future risk) against exploitation (making direct progress toward the sample-collection goal) under significant uncertainty and time/power budgets.

Task 3:

The rover maintains an internal representation of the terrain it has traversed — typically a grid or graph where each cell/node is tagged as free, hazardous, or unknown, along with an estimated traversal cost. As sensors gather new data at each step, this model is updated: previously unknown cells become known, cost estimates are refined based on actual difficulty encountered, and heuristic values for unexplored regions are revised. This is analogous to Learning Real-Time A* (LRTA*), where the agent updates the heuristic estimate of a state once it has actually experienced the true cost of reaching/leaving it, so that future decisions from that state (or ones like it) are better informed and the rover avoids repeating past mistakes.

Task 4:

Uninformed offline search (e.g., BFS, DFS) requires a fully known state graph in advance and has no way to incorporate a full Martian map that doesn't exist — it simply cannot be applied as-is. Informed offline search (e.g., A*) could, in principle, produce an optimal path, but only if a complete map and reliable heuristic were available beforehand, which contradicts the scenario's unknown-terrain premise. Online search agents (e.g., LRTA*) instead operate with only local, partial knowledge, learning and refining their model as they physically explore; they typically sacrifice a guarantee of finding the globally optimal path on the first attempt in exchange for the ability to function safely and adaptively when no prior map exists — which is the defining requirement of this scenario.

Task 5:

An online search agent (an LRTA*-style approach) is the most suitable choice. It directly addresses partial observability by only requiring local sensing at each step, adapts naturally to dynamic terrain changes by continuously updating its internal model rather than trusting a static plan, and improves safety because decisions are always grounded in the rover's most current, directly observed information rather than stale or incomplete offline data. While it may not guarantee the shortest possible path on the first traversal, this trade-off is justified given that safety and adaptability far outweigh strict path optimality in an unknown, hazardous, communication-delayed environment such as the Martian surface.

Scenario 4:

Task 1:

Variables: one variable per exam/course to be scheduled ($X_1, X_2, \dots, X_n$), each representing 'when and where this exam takes place'. Domains: for each exam variable, the domain is the set of feasible (time slot, exam hall) pairs — e.g., {(Mon 9am, Hall A), (Mon 9am, Hall B), (Mon 2pm, Hall A), ...}. Constraints: (1) No-overlap constraint — for any two exams sharing at least one common student, their assigned time slots must differ (a binary constraint between those exam variables); (2) Hall-capacity constraint — for any given time slot, the number of exams assigned to run simultaneously cannot exceed the number of available halls, and no two exams can be assigned the same hall in the same slot; (3) Precedence constraint — for subject pairs where one must be scheduled before another, the assigned time slot of the prerequisite exam must occur earlier than the dependent exam's slot; (4) Faculty-availability constraint — an exam's slot must fall within the available time windows of the faculty

required to invigilate it; (5) Accommodation constraint — students needing special arrangements (e.g., extended time or a separate room) restrict the domain of their associated exam(s) to only slot/hall combinations that satisfy those needs.

Task 2:

Backtracking search assigns a (time slot, hall) value to one exam at a time, checking after each assignment whether it is consistent with all constraints involving previously assigned exams (no shared-student overlap, no hall double-booking, precedence respected, faculty available, accommodations satisfied). If an assignment leads to a conflict with no valid options remaining, the algorithm backtracks to the previous exam and tries a different value from its domain. This continues until either a complete, consistent timetable is found or the search space is exhausted. Efficiency is typically improved using variable-ordering heuristics such as Minimum Remaining Values (schedule the most constrained exam — e.g., one shared by the most students or with tight faculty availability — first) and value-ordering heuristics such as Least Constraining Value.

Task 3:

Forward checking, after each exam is assigned a slot/hall, immediately removes now-inconsistent values from the domains of unassigned exams that share a constraint with it (for example, removing that slot from the domain of any other exam sharing a common student, or removing that specific hall/slot combination from exams that would clash there). This lets the algorithm detect dead ends much earlier — before deep, wasted backtracking occurs — because a domain becoming empty immediately signals failure. Stronger propagation techniques such as AC-3 (arc consistency) can further tighten domains by enforcing consistency between every pair of constrained variables, pruning even more of the search space before backtracking search needs to explore it, which is critical as the number of courses and students grows.

Task 4:

As the university scales up, the number of variables (exams) and the size of each domain (slot/hall combinations) grow rapidly, and constraints become denser (more shared students, more faculty overlaps), causing pure backtracking to become slow due to combinatorial explosion. The most scalable strategy combines backtracking search with constraint propagation (forward checking or arc consistency) to prune invalid branches early, strong variable/value-ordering heuristics (MRV, degree heuristic, least-constraining value) to reduce the number of backtracks needed, and — for very large institutions where finding a perfect assignment is too slow — a local-search repair method such as min-conflicts (start from a complete but possibly inconsistent assignment and iteratively resolve the exam with the most conflicts) to quickly reach a near-feasible schedule. This hybrid of systematic CSP techniques with heuristic repair offers the best balance of completeness and scalability for real-world timetabling.

Scenario 5:

Task 1:

Minimax models the game as a tree of alternating turns between a MAX player (the AI, trying to maximize its outcome) and a MIN player (the human opponent, assumed to play optimally

to minimize the AI's outcome). Utility values at terminal (or cutoff) states are propagated back up the tree: at MAX nodes the algorithm chooses the child with the highest value, and at MIN nodes it chooses the child with the lowest value, reflecting the assumption of a rational, worst-case-optimal adversary. This produces a move for the AI that guarantees the best achievable outcome even against the opponent's best possible counterplay — an appropriate model for the zero-sum, adversarial nature of a competitive strategy game.

Task 2:

The size of a full game tree grows as $O(b^d)$, where b is the branching factor (number of legal moves per turn) and d is the search depth. In a real-time strategy game, the branching factor can be very large (many units, many possible actions each), so even a modest search depth becomes computationally prohibitive to explore exhaustively within the strict per-decision time limit the AI is given. This makes it infeasible to search all the way to terminal (game-over) states, forcing the use of depth-limited search combined with a heuristic evaluation function at the cutoff depth, plus pruning techniques to avoid wasting time on branches that cannot influence the final decision.

Task 3:

Alpha-Beta pruning augments Minimax with two bounds while traversing the tree: alpha (the best value the MAX player can guarantee so far along the current path) and beta (the best value the MIN player can guarantee so far). While exploring a node's children, if at any point alpha becomes greater than or equal to beta, the remaining children of that node can be safely skipped ('pruned'), because it is already proven that a rational opponent would never allow play to reach that branch — it can't change the final decision at the root. Crucially, this produces exactly the same result as full Minimax, just faster. With good move ordering (evaluating the most promising moves first), Alpha-Beta pruning achieves a best-case time complexity of $O(b^{(d/2)})$, effectively allowing the AI to search twice as deep within the same time budget compared to plain Minimax.

Task 4:

Since exhaustive search to terminal states is infeasible, a heuristic evaluation function estimates how favorable a non-terminal game state is for the AI. Suitable factors include: material/unit advantage (relative strength and number of units/resources compared to the opponent), positional control (territory held, strategic map points occupied), economic state (resource-gathering rate, production capacity), mobility (number of available/legal moves, reflecting flexibility), threat level (how exposed key assets are to enemy attack), and tempo (initiative — whether the AI is forcing the opponent to react). These are combined, typically as a weighted sum (or a learned function, e.g., via reinforcement learning, if higher accuracy is needed), because each factor correlates with the likelihood of eventually winning and can be computed quickly at the cutoff depth without needing to search further.

Task 5:

The recommended strategy is iterative deepening Alpha-Beta search bounded by the available time limit: the AI searches to depth 1, then 2, then 3, and so on, always retaining the best move found so far so a valid decision is ready the instant time runs out. This is combined with strong move-ordering heuristics (such as killer-move and history heuristics) to maximize the effectiveness of Alpha-Beta pruning, and a transposition table to cache and reuse evaluations of previously seen game states, avoiding redundant computation. This approach deliberately trades a guarantee of globally optimal play for a strong, reliably timely decision — an

appropriate and necessary balance for a real-time strategy game opponent operating under strict per-move time limits.